Deterministic-Memory Streaming Anomaly Detection in Multivariate Telemetry via Phase-Space Delay Embedding and Index-Backed Random Cut Forests

Christos Iliakis
Department of Electrical and Computer Engineering
University of Western Macedonia
Kozani, Western Macedonia, Greece
ece02538@uowm.gr

***Abstract*—Real-time anomaly detection over high-frequency multivariate telemetry streams is a critical requirement for autonomous cyber-physical systems, avionics flight computers, and distributed computing infrastructure. Existing unsupervised streaming detectors suffer from two fundamental limitations when deployed in latency-critical environments: first, point-wise spatial partitioning algorithms fail to isolate temporal, frequency, and cross-channel phase anomalies without explicit dynamical state reconstruction; second, conventional tree-based data structures rely on dynamic heap memory allocation during sliding-window insertions and evictions, introducing non-deterministic allocator lock contention, memory fragmentation, and severe tail-latency jitter. This paper presents a strictly zero-heap-allocation streaming anomaly detection architecture that unifies online moment standardization, Takens' phase-space delay coordinate embedding, an index-addressed Robust Random Cut Forest (RRCF), and an exponentially decayed merging t-digest quantile sketch over a wait-free Single-Producer Single-Consumer (SPSC) shared-memory ring buffer. By exploiting the structural property that a full binary tree over a sliding window of W leaves contains at most $2W - 1$ vertices, we replace virtual memory pointers with 32-bit array offsets inside a pre-allocated contiguous slab arena, achieving $O(1)$ worst-case node allocation and deallocation without runtime heap invocations. Furthermore, we introduce a single-pass top-down iterative tree insertion algorithm with strict floating-point boundary invariants, scale-normalized Collusive Displacement (CoDisp) scoring during window warm-up, and anomaly contamination gating for tail quantile tracking. Theoretical analysis and empirical verification confirm that the proposed framework maintains a strictly bounded memory footprint, preserves exact binary space partitioning invariants under degenerate duplicate streams, and achieves sub-10 μs tail latency per multivariate telemetry frame. The complete open-source implementation is available at GitHub Repository.**

***Index Terms*—Anomaly detection, autonomous avionics, delay coordinate embedding, lock-free synchronization, quantile sketching, random cut forests, real-time systems, streaming algorithms.**

I. Introduction

MODERN autonomous cyber-physical systems—ranging from unmanned aerial vehicles (UAVs) and launch vehicle avionics to industrial robotics and high-throughput server clusters—generate continuous, high-rate multivariate telemetry streams [1], [2]. In safety-critical operational regimes, the timely identification of anomalous subsystem behavior—such as control-surface actuator jamming, parasitic structural flutter, inertial sensor bias drift, or correlated resource exhaustion—is essential for fault-tolerant control and mission assurance. Because exhaustive prior examples of failure modes are rarely available in aerospace and robotic deployments, anomaly detection engines must operate in a fully unsupervised, online streaming manner while adapting continuously to non-stationary baseline shifts.

Despite significant advances in unsupervised outlier detection, deploying streaming anomaly detectors onto onboard real-time processors exposes a sharp tension between *algorithmic expressiveness* and *systems-level determinism*:

1) **Inability of Point Detectors to Capture Dynamical Regime Shifts:** Classical spatial isolation structures, such as the Isolation Forest [3] and Half-Space Trees [4], evaluate each incoming measurement vector $x_t \in \mathbb{R}^d$ as an independent point in Euclidean space. When an anomaly manifests as a change in oscillation frequency, damping ratio, or cross-channel phase coupling rather than an instantaneous amplitude excursion, individual observations remain inside the nominal marginal support and evade detection.
2) **Non-Deterministic Tail Latency from Dynamic Heap Allocation:** Streaming tree ensembles, notably the Robust Random Cut Forest (RRCF) [5], adapt to evolving data distributions by continuously inserting new leaf nodes and evicting expired leaf nodes over a sliding window of size W. In standard pointer-based implementations, every sliding-window step invokes system heap allocators and deallocators across tens of trees. Over prolonged high-frequency operation, dynamic heap management induces virtual memory fragmentation, cache-

The complete open-source implementation and verification suite are publicly available at: GitHub Repository.

locality degradation across non-contiguous pages, and operating-system allocator lock contention, resulting in severe 99.9th percentile (p99.9) tail-latency spikes that violate hard real-time execution budgets.

3) **Threshold Freezing and Contamination in Infinite Streams:** Translating continuous anomaly scores into binary fault alarms requires tracking extreme tail quantiles (e.g., the 99.5th percentile) in bounded memory. Naive streaming quantile estimators either accumulate unbounded historical weight—rendering the threshold unresponsive to legitimate thermal or operational drift—or ingest anomalous score excursions directly into the baseline summary, inflating the decision threshold until the detector becomes blind to persistent faults.

To resolve these challenges simultaneously, this work introduces a unified, deterministic-memory streaming telemetry sketching and anomaly detection framework. Rather than treating algorithmic design and low-level memory architecture as separate concerns, we co-design the mathematical estimation pipeline around cache-aligned, pre-allocated static memory primitives.

The primary contributions of this paper are summarized as follows:

- **Phase-Space Shingled RRCF Pipeline:** We couple online Welford Z-score standardization [6] with Takens' time-delay coordinate embedding [7], [8] prior to randomized axis-aligned space partitioning. Lifting d-dimensional telemetry into an $(m \cdot d)$-dimensional reconstructed phase space transforms temporal derivatives and frequency shifts into spatial deviations off the nominal attractor manifold.
- **Zero-Allocation Index-Backed Slab Tree Architecture:** Exploiting the combinatorial invariant that a full binary tree of W leaves contains exactly $W - 1$ internal nodes, we design a contiguous slab arena per tree of capacity $2W$ addressed via 32-bit integer indices. We formulate a single-pass top-down iterative insertion algorithm and a constant-lookup bottom-up deletion algorithm that execute with strictly zero dynamic heap allocations in the steady-state loop.
- **Degenerate-Span and Floating-Point Boundary Robustness:** We identify and eliminate a subtle IEEE-754 floating-point boundary inversion hazard in randomized cut partitioning and establish a deterministic leaf-splitting rule for zero-span duplicate vectors, guaranteeing a strict bijection between sliding-window slots and leaf indices without hash tables or reference counting.
- **Decayed Adaptive Tail Thresholding and Wait-Free Ingestion:** We pair scale-normalized Collusive Displacement scoring with an exponentially decayed, contamination-gated merging t-digest [9], fed by a wait-free Single-Producer Single-Consumer (SPSC) shared-memory ring buffer [11] with local index shadow caching to eliminate inter-core false sharing.

The remainder of this manuscript is organized as follows. Section II reviews related literature in streaming outlier detection, phase-space reconstruction, quantile sketching, and real-time memory management. Section III formalizes the streaming telemetry model and phase-space delay embedding. Section IV details the wait-free inter-process ingestion layer and the index-backed slab memory model. Section V presents the core algorithms for dynamic Random Cut Tree maintenance, scale-normalized Collusive Displacement scoring, and decayed t-digest thresholding. Section VI provides formal proofs of space complexity, time complexity, and structural tree invariants. Section VII describes the verification methodology and empirical performance across synthetic fault regimes and real-world avionics benchmarks. Finally, Section VIII concludes the paper.

II. Related Work

A. Unsupervised Streaming Anomaly Detection

Tree-based spatial isolation was pioneered by Liu et al. [3] with the Isolation Forest (iForest), which demonstrated that anomalous points are isolated closer to the root of randomized binary space partitioning trees than nominal points. However, iForest selects split dimensions uniformly at random $(1/D)$ and requires offline batch reconstruction, making it unsuitable for non-stationary streams or high-dimensional spaces containing irrelevant noisy channels. Tan et al. [4] proposed Streaming Half-Space Trees (HS-Trees), which construct fixed-depth full binary trees over an initial reference window and update node mass profiles across tumbling windows, but HS-Trees cannot adapt their spatial partition boundaries once initialized.

Guha et al. [5] overcame this limitation by introducing the Robust Random Cut Forest (RRCF). By choosing split dimensions with probability proportional to the side lengths of each node's minimal enclosing axis-aligned bounding box, RRCF ensures sensitivity to geometric scale while preserving a key distributional invariant: dynamically inserting and deleting points produces a tree whose probability distribution is identical to a batch tree constructed from scratch on the active sample set. Guha et al. further introduced Collusive Displacement (CoDisp) to account for masking effects when outliers arrive in small bursts. Deep learning alternatives, such as Long Short-Term Memory (LSTM) prediction-error models proposed by Hundman et al. [2] for spacecraft telemetry, achieve strong expressive power on complex sequences but require extensive offline training and incur computational overheads that challenge microsecond-scale onboard budgets.

B. Phase-Space Reconstruction in Dynamical Systems

The geometric reconstruction of nonlinear dynamical systems from time-series observations rests on Takens' delay embedding theorem [7] and its generalization by Sauer et al. [8]. Given a trajectory evolving on a compact manifold $\mathcal{M}$ of dimension $d_{\mathcal{M}}$, delay coordinate maps of dimension $m > 2d_{\mathcal{M}}$ form a diffeomorphic embedding of the underlying attractor. In streaming signal processing, concatenating m consecutive or strided multivariate vectors—often termed *shingling* [5]—projects temporal autocorrelation structure and cross-sensor phase relationships into a single joint Euclidean vector space.

C. Streaming Quantile Sketches and Real-Time Systems

Maintaining adaptive decision thresholds over unbounded streams requires sub-linear space quantile summaries. While deterministic summaries such as the Greenwald-Khanna (GK) algorithm [12] provide uniform ϵ-approximate rank guarantees across $[0, 1]$, anomaly detection requires extreme precision specifically in the upper tail ($q \geq 0.99$). Dunning and Ertl [9], [10] introduced the t-digest, which replaces uniform error bounds with a non-linear arcsine scale function that forces cluster centroids near $q \in \{0, 1\}$ to maintain minimal weight, yielding part-per-million accuracy in the tails within constant memory. On the systems side, predictable real-time execution builds upon Bonwick's slab allocation paradigm [13] and Lamport's lock-free concurrent ring buffer theory [11], which eliminate operating-system kernel traps and mutex serialization in the critical data path.

III. Preliminaries and Problem Formulation

A. Streaming Telemetry Model and Online Standardization

Let $\mathcal{S} = \{(x_t, \tau_t)\}_{t=1}^{\infty}$ denote an unbounded, discrete-time multivariate telemetry stream, where $x_t = [x_{t,1}, x_{t,2}, \ldots, x_{t,d}]^\top \in \mathbb{R}^d$ represents d synchronized sensor channels sampled at monotonic timestamp $\tau_t \in \mathbb{R}_{\geq 0}$.

In physical telemetry systems, individual channels span disparate orders of magnitude (e.g., barometric static pressure in Pascals versus angular roll rates in radians per second). Because a Random Cut Tree selects split coordinates proportional to the geometric side lengths of its axis-aligned bounding boxes, unnormalized channels with large numerical ranges dominate the partitioning probability distribution. To enforce scale invariance across channels in $O(1)$ space and time per sample, each raw observation x_t is standardized via Welford's numerically stable recurrence [6]. For channel $i \in \{1, \ldots, d\}$ at time step $t \leq W_{\text{init}}$:

$$\delta_{t,i} = x_{t,i} - \mu_{t-1,i}, \tag{1}$$

$$\mu_{t,i} = \mu_{t-1,i} + \frac{\delta_{t,i}}{t}, \tag{2}$$

$$M_{2,t,i} = M_{2,t-1,i} + \delta_{t,i}\left(x_{t,i} - \mu_{t,i}\right), \tag{3}$$

$$\sigma_{t,i} = \sqrt{\frac{M_{2,t,i}}{\max(1, t-1)} + \epsilon_\sigma}, \tag{4}$$

where $\epsilon_\sigma = 10^{-8}$ prevents division by zero on constant signals. The standardized vector $\tilde{x}_t \in \mathbb{R}^d$ is given componentwise by $\tilde{x}_{t,i} = (x_{t,i} - \mu_{t,i})/\sigma_{t,i}$. To prevent severe faults after deployment from corrupting the standardization moments, the moments $(\mu_{t,i}, \sigma_{t,i})$ may optionally be frozen at $t = W_{\text{init}}$.

B. Takens' Phase-Space Delay Embedding

Suppose the underlying cyber-physical system evolves according to a latent state $\xi_t \in \mathcal{M}$ on a smooth dynamical manifold $\mathcal{M}$. By Takens' embedding theorem [7], the local phase-space trajectory can be reconstructed from a finite sequence of time-delayed observations.

Definition 1 (Multivariate Delay Coordinate Map)**.** *Given embedding dimension (shingle size) $m \in \mathbb{N}_{\geq 1}$ and delay stride $\eta \in \mathbb{N}_{\geq 1}$, let $L = (m-1)\eta + 1$ be the required history horizon. For all $t \geq L$, the delay coordinate embedding operator $\Phi_{m,\eta} : (\mathbb{R}^d)^L \to \mathbb{R}^D$ with $D = m \cdot d$ constructs the shingled state vector:*

$$v_t = \left[\tilde{x}_t^\top, \quad \tilde{x}_{t-\eta}^\top, \quad \tilde{x}_{t-2\eta}^\top, \quad \ldots, \quad \tilde{x}_{t-(m-1)\eta}^\top\right]^\top \in \mathbb{R}^D. \tag{5}$$

When $m = 1$, $v_t = \tilde{x}_t$ captures only instantaneous spatial position. When $m > 1$, linear combinations of the coordinates of v_t span finite-difference approximations of first- and higher-order temporal derivatives up to order $m-1$, as well as cross-channel phase lags. Consequently, an anomalous frequency shift or structural oscillation causes v_t to depart orthogonally from the nominal attractor manifold in $\mathbb{R}^D$, even if $\|\tilde{x}_t\|_\infty$ remains bounded within normal marginal limits.

C. Real-Time Streaming Anomaly Detection Objective

Given the embedded stream $\{v_t\}_{t=L}^{\infty}$ over a sliding window of capacity W, our objective at each step t is to compute a scalar anomaly score $\tilde{C}_t \in \mathbb{R}_{\geq 0}$ and an adaptive tail threshold $\Theta_t(\alpha)$ corresponding to quantile $\alpha \in (0, 1)$ (typically $\alpha = 0.995$), emitting an alarm indicator $A_t = \mathbb{I}(\tilde{C}_t > \Theta_t(\alpha))$ subject to two strict systems constraints:

1) **Zero-Heap-Allocation Constraint:** The number of dynamic memory allocations invoked on the operating system heap during processing of frame t must equal zero for all $t \geq 1$.
2) **Bounded Memory Constraint:** Total memory consumption is bounded by a deterministic constant $\mathcal{M}_{\max}(d, m, B, W, C)$ fixed at initialization, where B is the number of trees in the forest and C is the maximum centroid capacity of the quantile sketch.

IV. System Architecture and Deterministic Memory Model

Fig. 1 illustrates the complete pipeline architecture. Below, we formalize the two foundational systems mechanisms that guarantee deterministic latency: wait-free inter-process ingestion and index-addressed contiguous slab allocation.

A. Wait-Free SPSC Shared-Memory Ingestion with Shadow Index Caching

To decouple high-frequency sensor acquisition or log replay from anomaly detection across process boundaries without kernel context switches, telemetry frames are transported through a POSIX shared memory segment mapped into the virtual address spaces of both the producer and consumer processes.

Let R be a power-of-two ring buffer capacity ($R = 2^k$), allowing modulo arithmetic to be replaced by a single bitwise conjunction $i \bmod R = i \wedge (R-1)$. In a standard concurrent ring buffer [11], the producer increments an atomic write index w and reads an atomic read index r, while the consumer increments r and reads w. On modern multicore processors implementing the MESI (Modified, Exclusive, Shared, Invalid) cache-coherency protocol, two hardware bottlenecks arise if w and r are naively accessed on every frame:

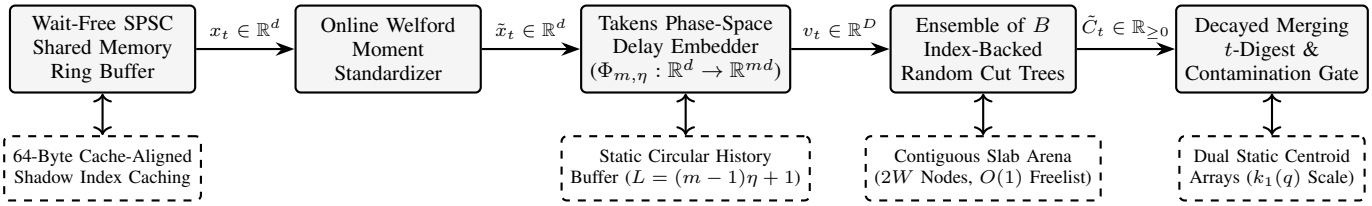


Fig. 1. System architecture of the proposed deterministic-memory streaming anomaly detection pipeline. Solid blocks denote sequential hot-path transformations; dashed blocks indicate the underlying pre-allocated static memory structures that guarantee zero dynamic heap allocation.

1) **False Sharing:** If w and r reside on the same 64-byte hardware cache line, every store by the producer invalidates the consumer's L1 cache line and vice versa.
2) **Redundant Cross-Core Cache Line Transfers:** Even when placed on separate cache lines, reading the remote thread's atomic index on every operation forces a cache coherency transaction whenever the remote thread advances.

We eliminate both bottlenecks simultaneously. First, the shared memory segment is structured with strict 64-byte cache-line alignment:

- Cache Line 0: An atomic 64-bit initialization readiness token M_{magic} that prevents cross-process startup race conditions (the consumer spins on an acquire load until the producer completes shared-memory mapping and releases M_{magic}).
- Cache Line 1: The atomic write index w and the producer-private shadow read cache $\hat{r}_p$.
- Cache Line 2: The atomic read index r and the consumer-private shadow write cache $\hat{w}_c$.
- Cache Lines ≥ 3: The contiguous frame array $\mathcal{B}[0 \dots R-1]$.

When the producer attempts to enqueue a frame, it loads its own write index w using *relaxed* memory ordering and checks whether $w - \hat{r}_p < R$. Because the consumer can only increase r monotonically, $w - \hat{r}_p < R$ is a sufficient condition guaranteeing $w - r < R$. The producer only performs an *acquire* load on the atomic variable r (updating $\hat{r}_p \leftarrow r$) when the local shadow check indicates the buffer might be full ($w - \hat{r}_p = R$). Symmetrically, the consumer only performs an *acquire* load on w (updating $\hat{w}_c \leftarrow w$) when $\hat{w}_c = r$. Consequently, under normal operating load ($0 < w - r < R$), inter-core cache coherency traffic on the control indices is reduced by up to two orders of magnitude.

B. Index-Addressed Contiguous Slab Pool

In a dynamic binary tree, each sliding-window insertion creates a new leaf node and a new internal parent node, while each eviction removes an expired leaf node and its immediate parent. To eliminate heap allocation entirely, we exploit a structural property of full binary trees.

Lemma 1 (Vertex Cardinality of a Random Cut Tree)**.** *Let T be a Random Cut Tree containing $n \geq 1$ leaf nodes. Because every internal split partitions a non-empty subset of points into two non-empty child subtrees, T is a strict full binary tree (every internal node has out-degree exactly 2). Therefore, T contains exactly $n-1$ internal nodes and $2n-1$ total nodes. For a sliding window of maximum capacity W, the total number of active nodes in T satisfies $|V(T)| \leq 2W-1 < 2W$ at all times.*

By Lemma 1, each tree T_b ($b \in \{1, \dots, B\}$) is backed by a fixed-capacity contiguous slab array $\mathcal{P}_b[0 \dots 2W-1]$ allocated once at initialization. Rather than linking tree nodes via 64-bit virtual memory pointers (which consume 24 bytes per node for parent, left, and right links and cannot be relocated in memory), all tree topology edges are encoded as 32-bit unsigned integer array indices $u \in \{0, 1, \dots, 2W-1\}$, with a reserved sentinel constant $\varnothing = 2^{32} - 1$ denoting a null link.

Free slots within $\mathcal{P}_b$ are managed via an $O(1)$ intrusive freelist array $F[0 \dots 2W-1]$ and a scalar head index h_{free} initialized as $F[i] = i+1$ for $0 \leq i < 2W-1$, $F[2W-1] = \varnothing$, and $h_{\text{free}} = 0$:

- **Allocation (Allocate()):** Returns $u \leftarrow h_{\text{free}}$, updates $h_{\text{free}} \leftarrow F[u]$, initializes $\mathcal{P}_b[u]$, and increments the active node counter in $O(1)$ worst-case time.
- **Deallocation (Deallocate(u)):** Sets $F[u] \leftarrow h_{\text{free}}$, updates $h_{\text{free}} \leftarrow u$, and decrements the active node counter in $O(1)$ worst-case time.

V. Streaming Algorithmic Framework

A. Dynamic Robust Random Cut Tree Maintenance

Each node $u \in \mathcal{P}_b$ in a Random Cut Tree over $\mathbb{R}^D$ stores five fields:

1) The minimal enclosing axis-aligned bounding box $B(u) = \prod_{i=1}^{D} [\ell_i(u), h_i(u)]$, represented by lower and upper coordinate vectors $\ell(u), h(u) \in \mathbb{R}^D$.
2) The subtree leaf mass $M(u) \in \{1, \dots, W\}$.
3) Topological indices $\text{parent}(u), \text{left}(u), \text{right}(u) \in \{0, \dots, 2W-1\} \cup \{\varnothing\}$.
4) For internal nodes, the split dimension $q(u) \in \{1, \dots, D\}$ and cut threshold $c(u) \in \mathbb{R}$.

1) Single-Pass Top-Down Iterative Insertion: When inserting a new shingled point $p \in \mathbb{R}^D$ into a tree rooted at r_{root}, standard recursive formulations traverse down the tree and subsequently unwind the call stack to update ancestor bounding boxes. We formulate a single-pass top-down iterative procedure (Algorithm 1) that updates bounding boxes and subtree masses *in-place during descent*.

At the current node u, we compute the expanded bounding box $\hat{B}(u) = B(u) \cup \{p\}$ with coordinates $\hat{\ell}_i = \min(\ell_i(u), p_i)$ and $\hat{h}_i = \max(h_i(u), p_i)$, along with directional spans $s_i = \hat{h}_i - \hat{\ell}_i$ and total span $S_{\text{total}} = \sum_{i=1}^{D} s_i$. A random variable $r \sim$

Uniform$(0, S_{\text{total}})$ simultaneously selects the split coordinate $q \in \{1, \ldots, D\}$ with probability s_q / S_{total} and a cut location $c \in [\hat{\ell}_q, \hat{h}_q]$.

If the proposed cut c strictly separates p from $B(u)$—that is, $c < \ell_q(u)$ or $c > h_q(u)$—or if u is already a leaf node, the algorithm terminates its descent immediately, allocates a new leaf node u_{leaf} and a new internal parent node u_{par} from the slab pool, splices u_{par} in place of u, and returns u_{leaf}. Otherwise, $c \in [\ell_q(u), h_q(u)]$, which implies that the existing split at u takes precedence over the new cut; because p is now guaranteed to be inserted within the subtree rooted at u, we update $\ell(u) \leftarrow \hat{\ell}$, $h(u) \leftarrow \hat{h}$, and $M(u) \leftarrow M(u) + 1$ immediately and step down to left(u) if $p_{q(u)} \le c(u)$ or right(u) otherwise.

2) Resolution of Two Numerical Edge Cases: Algorithm 1 resolves two critical edge cases that corrupt naive implementations:

- **IEEE-754 Floating-Point Boundary Inversion:** When $p_q > h_q(u)$ and a separating cut $c \in [h_q(u), p_q]$ is drawn, finite-precision floating-point addition $c = \hat{\ell}_q + \Delta$ can round c to equal p_q exactly. If child assignment were decided by testing $p_q \le c$, the new leaf u_{leaf} (containing the larger coordinate p_q) would be placed as the *left* child while the existing subtree u (containing strictly smaller coordinates $\le h_q(u)$) would be placed as the *right* child, violating the binary space partitioning invariant. Line 25 of Algorithm 1 eliminates this hazard by testing $p_q \le \ell_q(u)$ directly against the existing bounding box.
- **Degenerate Zero-Span Duplicate Streams ($S_{\text{total}} \le 10^{-12}$):** When a sensor saturates or freezes, consecutive shingled vectors may be identical ($p = \ell(u) = h(u)$), yielding $S_{\text{total}} = 0$. Rather than coalescing duplicate points into a single leaf with reference counts and hash tables, Lines 14–15 force a deterministic split whenever traversal reaches an existing leaf. This preserves a strict bijection between the W sliding-window slots and W distinct leaf indices in $\mathcal{P}_b$.

3) Constant-Lookup Eviction and Bottom-Up Tightening: Because each sliding-window slot $s \in \{0, \ldots, W-1\}$ stores the exact leaf index u_{leaf} returned upon insertion, evicting the oldest point requires no spatial search (Algorithm 2). Given u_{leaf}, its parent $u_{\text{par}} = \text{parent}(u_{\text{leaf}})$ and sibling u_{sib} are identified in $O(1)$ time. The sibling u_{sib} is promoted directly into the grandparent $u_{\text{grand}} = \text{parent}(u_{\text{par}})$, both u_{leaf} and u_{par} are returned to the slab freelist, and the algorithm walks upward from u_{grand} to r_{root}, tightening each ancestor's bounding box and mass exactly from its two children:

$$M(v) = M(\text{left}(v)) + M(\text{right}(v)), \tag{6}$$

$$\ell_i(v) = \min\big(\ell_i(\text{left}(v)), \ell_i(\text{right}(v))\big), \quad \forall i \in \{1, \ldots, D\}, \tag{7}$$

$$h_i(v) = \max\big(h_i(\text{left}(v)), h_i(\text{right}(v))\big), \quad \forall i \in \{1, \ldots, D\}. \tag{8}$$

B. Scale-Normalized Collusive Displacement Scoring

In shingled telemetry streams, a transient fault persists in the embedded vector v_t across m consecutive steps, creating a small cluster of mutually masking outlier points. To prevent masking, we evaluate each newly inserted leaf u_{leaf} using Collusive Displacement (CoDisp) [5], which measures the maximum ratio of sibling subtree mass to current subtree mass along the path $\mathcal{P}(u_{\text{leaf}} \to r_{\text{root}})$:

$$\text{CoDisp}(u_{\text{leaf}}, T_b) = \max_{u \in \mathcal{P}(u_{\text{leaf}} \to r_{\text{root}}) \setminus \{r_{\text{root}}\}} \frac{M(\text{sibling}(u))}{M(u)}. \tag{9}$$

Algorithm 1 Single-Pass Iterative Point Insertion in Index-Backed RRCT

Require: Point $p \in \mathbb{R}^D$, Slab Pool $\mathcal{P}$, Root Index r_{root}, PRNG $\mathcal{G}$
Ensure: Allocated leaf index u_{leaf}

1: **if** $r_{\text{root}} = \varnothing$ **then**
2: $u_{\text{leaf}} \leftarrow \mathcal{P}$.Allocate()
3: $\ell(u_{\text{leaf}}) \leftarrow p, \quad h(u_{\text{leaf}}) \leftarrow p, \quad M(u_{\text{leaf}}) \leftarrow 1$
4: $r_{\text{root}} \leftarrow u_{\text{leaf}}$
5: **return** u_{leaf}
6: **end if**
7: $u \leftarrow r_{\text{root}}$
8: **loop**
9: $S_{\text{total}} \leftarrow 0$
10: **for** $i = 1$ **to** D **do**
11: $\hat{\ell}_i \leftarrow \min(\ell_i(u), p_i), \quad \hat{h}_i \leftarrow \max(h_i(u), p_i)$
12: $s_i \leftarrow \hat{h}_i - \hat{\ell}_i, \quad S_{\text{total}} \leftarrow S_{\text{total}} + s_i$
13: **end for**
14: **if** $S_{\text{total}} \le 10^{-12}$ **then**
15: $q \leftarrow 1, \quad c \leftarrow p_1, \quad \text{separates} \leftarrow \text{IsLeaf}(u)$
16: **else**
17: Draw $r \sim \text{Uniform}(0, S_{\text{total}})$ using $\mathcal{G}$
18: Select dimension q and offset $\Delta \in [0, s_q]$ where $\sum_{j<q} s_j \le r \le \sum_{j \le q} s_j$
19: $c \leftarrow \ell_q + \Delta$
20: $\text{separates} \leftarrow \text{IsLeaf}(u) \vee (c < \ell_q(u)) \vee (c > h_q(u))$
21: **end if**
22: **if** separates **then**
23: $u_{\text{leaf}} \leftarrow \mathcal{P}$.Allocate(), $\quad u_{\text{par}} \leftarrow \mathcal{P}$.Allocate()
24: $\ell(u_{\text{leaf}}) \leftarrow p, \quad h(u_{\text{leaf}}) \leftarrow p, \quad M(u_{\text{leaf}}) \leftarrow 1, \quad \text{parent}(u_{\text{leaf}}) \leftarrow u_{\text{par}}$
25: $\ell(u_{\text{par}}) \leftarrow \hat{\ell}, \quad h(u_{\text{par}}) \leftarrow \hat{h}, \quad q(u_{\text{par}}) \leftarrow q, \quad M(u_{\text{par}}) \leftarrow M(u) + 1$
26: **if** $p_q \le \ell_q(u)$ **then**
27: $\text{left}(u_{\text{par}}) \leftarrow u_{\text{leaf}}, \quad \text{right}(u_{\text{par}}) \leftarrow u, \quad c(u_{\text{par}}) \leftarrow \min(c, \ell_q(u))$
28: **else**
29: $\text{left}(u_{\text{par}}) \leftarrow u, \quad \text{right}(u_{\text{par}}) \leftarrow u_{\text{leaf}}, \quad c(u_{\text{par}}) \leftarrow \max(c, h_q(u))$
30: **end if**
31: Splice u_{par} into parent(u) and update parent$(u) \leftarrow u_{\text{par}}$
32: **return** u_{leaf}
33: **end if**
34: $\ell(u) \leftarrow \hat{\ell}, \quad h(u) \leftarrow \hat{h}, \quad M(u) \leftarrow M(u) + 1$
35: $u \leftarrow \text{left}(u)$ if $p_{q(u)} \le c(u)$ else right(u)
36: **end loop**

Algorithm 2 Constant-Lookup Point Eviction and Bounding-Box Tightening

Require: Leaf Index u_{leaf}, Slab Pool $\mathcal{P}$, Root Index r_{root}
1: $u_{\text{par}} \leftarrow \text{parent}(u_{\text{leaf}})$
2: **if** $u_{\text{par}} = \varnothing$ **then**
3: $\quad \mathcal{P}.\text{Deallocate}(u_{\text{leaf}}), \quad r_{\text{root}} \leftarrow \varnothing$
4: $\quad$ **return**
5: **end if**
6: $u_{\text{sib}} \leftarrow \text{right}(u_{\text{par}})$ if $\text{left}(u_{\text{par}}) = u_{\text{leaf}}$ else $\text{left}(u_{\text{par}})$
7: $u_{\text{grand}} \leftarrow \text{parent}(u_{\text{par}})$
8: $\text{parent}(u_{\text{sib}}) \leftarrow u_{\text{grand}}$
9: **if** $u_{\text{grand}} = \varnothing$ **then**
10: $\quad r_{\text{root}} \leftarrow u_{\text{sib}}$
11: **else if** $\text{left}(u_{\text{grand}}) = u_{\text{par}}$ **then**
12: $\quad \text{left}(u_{\text{grand}}) \leftarrow u_{\text{sib}}$
13: **else**
14: $\quad \text{right}(u_{\text{grand}}) \leftarrow u_{\text{sib}}$
15: **end if**
16: $\mathcal{P}.\text{Deallocate}(u_{\text{leaf}}), \quad \mathcal{P}.\text{Deallocate}(u_{\text{par}})$
17: $v \leftarrow u_{\text{grand}}$
18: **while** $v \neq \varnothing$ **do**
19: $\quad u_L \leftarrow \text{left}(v), \quad u_R \leftarrow \text{right}(v)$
20: $\quad M(v) \leftarrow M(u_L) + M(u_R)$
21: $\quad$ **for** $i = 1$ **to** D **do**
22: $\quad\quad \ell_i(v) \leftarrow \min(\ell_i(u_L), \ell_i(u_R)), \quad h_i(v) \leftarrow \max(h_i(u_L), h_i(u_R))$
23: $\quad$ **end for**
24: $\quad v \leftarrow \text{parent}(v)$
25: **end while**

Intuitively, if a small colluding group of points u is isolated near the root of T_b, removing u reduces the depth of all $M(\text{sibling}(u))$ nominal leaves by 1, yielding a large displacement per removed point $M(\text{sibling}(u))/M(u)$.

During the initial warm-up phase when the sliding window contains $n_t < W$ active samples, the maximum attainable value of (9) is bounded above by $n_t - 1$. Feeding raw unscaled scores into the quantile sketch during warm-up biases the estimated tail distribution downward. We therefore apply a finite-window scale normalization across the ensemble of B trees:

$$\tilde{C}_t = \left(\frac{W}{n_t}\right) \frac{1}{B} \sum_{b=1}^{B} \text{CoDisp}\big(u_{\text{leaf}}^{(b)}, T_b\big). \tag{10}$$

C. Exponentially Decayed Merging t-Digest with Contamination Gating

To convert $\tilde{C}_t$ into a calibrated anomaly decision without hardcoded thresholds, the score stream is summarized by a zero-allocation merging t-digest [9]. The digest maintains $C \leq C_{\max}$ sorted 1D centroids $\{(m_j, w_j)\}_{j=1}^{C}$ (mean and weight) alongside a static incoming buffer of capacity U. Centroid granularity is governed by the arcsine scale function $k_1 : [0,1] \to [-\delta/4, \delta/4]$ with compression parameter $\delta > 0$:

$$k_1(q) = \frac{\delta}{2\pi} \arcsin(2q - 1). \tag{11}$$

Two adjacent centroids spanning cumulative weight quantile interval $[q_L, q_R]$ are merged if and only if:

$$k_1(q_R) - k_1(q_L) \leq 1. \tag{12}$$

Because $\frac{dk_1}{dq} = \frac{\delta}{\pi\sqrt{4q(1-q)}} \to \infty$ as $q \to 1$, centroids in the extreme upper tail ($q \geq 0.99$) are restricted to minimal weight, preserving sharp quantile accuracy for anomaly thresholding while aggressively compressing the bulk of the nominal score distribution ($q \in [0.1, 0.9]$).

To prevent infinite-stream weight saturation and anomaly threshold contamination, we introduce two modifications to the standard t-digest:

1) **Exponential Weight Decay:** Whenever the incoming buffer of size U fills and triggers a merge pass, the weights of all existing centroids are multiplied by a forgetting factor $\gamma \in (0,1)$ (e.g., $\gamma = 0.995$):
$$w_j \leftarrow \gamma w_j, \quad \forall j \in \{1, \ldots, C\}. \tag{13}$$
This bounds the effective historical weight of the sketch at steady state to $\lim_{k\to\infty} \sum_{s=0}^{k} U\gamma^s = \frac{U}{1-\gamma}$, allowing the adaptive threshold $\Theta_t(\alpha)$ to track gradual regime shifts over infinite streams.
2) **Anomaly Contamination Gate:** Once the sliding window is fully warmed up ($n_t = W$), a frame is flagged as anomalous if $\tilde{C}_t > \Theta_t(\alpha) = \max(\hat{Q}_{t-1}(\alpha), \theta_{\min})$. Crucially, if $A_t = 1$ (an anomaly is declared), $\tilde{C}_t$ is *withheld* from the t-digest buffer, preventing sustained fault excursions from corrupting the upper tail of the nominal score distribution.

Algorithm 3 synthesizes the complete end-to-end per-frame processing loop.

VI. Theoretical Complexity and Invariant Analysis

We now establish the formal memory, computational complexity, and structural correctness guarantees of the proposed architecture.

Theorem 1 (Strict Space Complexity and Zero-Heap Invariant)**.** *Consider a streaming detector configured with d channels, delay dimension m (with $D = md$ and stride η), ensemble size B, sliding window capacity W, and t-digest capacities $(C_{\max}, U)$. The total memory footprint is bounded by a deterministic constant:*

$$\mathcal{M}_{\textit{total}} = O\Big(d \cdot m\eta + B \cdot W \cdot D + C_{\max} + U\Big), \tag{14}$$

and the number of dynamic heap allocations invoked during processing of any sequence of N frames ($N \to \infty$) is strictly zero.

Proof. By Lemma 1, each tree T_b contains at most W leaves and $W - 1$ internal nodes. Because every eviction in Algorithm 3 (Line 7) deallocates exactly 2 slab indices (u_{leaf} and u_{par}) immediately prior to Line 9 allocating 2 slab indices, the active node count in $\mathcal{P}_b$ never exceeds $2W - 1$. Each node stores two D-dimensional bounding-box vectors and fixed scalar metadata, requiring $O(D)$ bytes, so all B slab pools

Algorithm 3 End-to-End Zero-Allocation Streaming Telemetry Pipeline

Require: Standardized Frame $\tilde{x}_t \in \mathbb{R}^d$, Forest $\{T_b\}_{b=1}^{B}$, Slot Matrix $S[1 \dots B][0 \dots W-1]$, Window Sampler $\mathcal{W}$, t-Digest $\mathcal{D}$, Cutoff α
Ensure: Anomaly Score $\tilde{C}_t$, Threshold Θ_t, Alarm Flag A_t
1: **if** $\neg\Phi_{m,\eta}$.PushAndEmbed$(\tilde{x}_t, v_t)$ **then**
2: **return** $(0, 0, \text{false})$ {Initial phase-space delay buffer filling}
3: **end if**
4: $(\text{evict}, s_{\text{idx}}) \leftarrow \mathcal{W}$.NextSlot()
5: $C_{\text{sum}} \leftarrow 0$
6: **for** $b = 1$ **to** B **do**
7: **if** evict **then**
8: DeletePoint$(S[b][s_{\text{idx}}], \mathcal{P}_b, r_{\text{root}}^{(b)})$ {Algorithm 2}
9: **end if**
10: $u_{\text{new}} \leftarrow$ InsertPoint$(v_t, \mathcal{P}_b, r_{\text{root}}^{(b)}, \mathcal{G})$ {Algorithm 1}
11: $S[b][s_{\text{idx}}] \leftarrow u_{\text{new}}$
12: $C_{\text{sum}} \leftarrow C_{\text{sum}} +$ CoDisp(u_{new}, T_b)
13: **end for**
14: $n_t \leftarrow \mathcal{W}$.ActiveCount()
15: $\tilde{C}_t \leftarrow \left(\frac{W}{n_t}\right) \frac{C_{\text{sum}}}{B}$
16: **if** $n_t = W$ **then**
17: $\Theta_t \leftarrow \max\left(\mathcal{D}.\text{Quantile}(\alpha), \theta_{\min}\right)$
18: $A_t \leftarrow (\tilde{C}_t > \Theta_t)$
19: **if** $\neg A_t$ **then**
20: $\mathcal{D}$.Add$(\tilde{C}_t)$ {Contamination-gated update}
21: **end if**
22: **else if** $n_t \geq W/2$ **then**
23: $\mathcal{D}$.Add$(\tilde{C}_t)$, $\Theta_t \leftarrow 0$, $A_t \leftarrow$ false
24: **end if**
25: **return** $(\tilde{C}_t, \Theta_t, A_t)$

occupy $O(B \cdot W \cdot D)$ static bytes. The delay embedder stores $L = (m-1)\eta + 1$ vectors in $\mathbb{R}^d$, requiring $O(d \cdot m\eta)$ bytes, and the t-digest uses fixed working arrays of size $2C_{\max} + U$. Since all state transitions operate exclusively via in-place array indexing over pre-allocated buffers, zero calls to dynamic system allocators occur for all $t \geq 1$. □

Theorem 2 (Per-Frame Time Complexity)**.** *Let $h(T_b)$ denote the height of Random Cut Tree T_b. Processing a single multivariate frame requires:*

$$\mathcal{T}_{\textit{frame}} = O\left(B \cdot D \cdot \bar{h} + \frac{(C_{\max} + U)\log(C_{\max} + U)}{U}\right) \tag{15}$$

amortized time, where $\bar{h} = \frac{1}{B}\sum_{b=1}^{B} h(T_b)$. Under non-adversarial distributions where expected tree depth satisfies $\mathbb{E}[\bar{h}] = O(\log W)$, expected per-frame processing time is $O(B \cdot D \log W)$ in the common case and $O(B \cdot D \cdot W)$ in the worst case.

Proof. Standardizing $\tilde{x}_t$ takes $O(d)$ time and constructing $v_t \in \mathbb{R}^D$ takes $O(D)$ time. For each of the B trees:

1) Point eviction (Algorithm 2) splices the sibling in $O(1)$ time and walks upward to the root, tightening D-dimensional bounding boxes at each ancestor in $O(D \cdot h(T_b))$ time.
2) Single-pass point insertion (Algorithm 1) performs $O(D)$ bounding-box span calculations at each visited ancestor and terminates in at most $h(T_b)$ steps, requiring $O(D \cdot h(T_b))$ time.
3) CoDisp evaluation walks the leaf-to-root path performing $O(1)$ mass divisions per ancestor in $O(h(T_b))$ time.

In the t-digest, buffer insertion takes $O(1)$ time for $U-1$ consecutive frames and triggers an $O((C_{\max}+U)\log(C_{\max}+U))$ sort-and-merge pass once every U frames. Summing across all stages yields the stated complexity. □

Proposition 1 (Preservation of Bounding-Box and Mass Invariants)**.** *At the conclusion of every insertion (Algorithm 1) and deletion (Algorithm 2), every internal node $v \in V(T_b)$ satisfies:*

$$M(v) = M(\textit{left}(v)) + M(\textit{right}(v)), \tag{16}$$

$$B(v) = B(\textit{left}(v)) \cup B(\textit{right}(v)). \tag{17}$$

Proof. We proceed by structural induction. For a single leaf root, the base case holds vacuously. Suppose (16)–(17) hold prior to inserting p. In Algorithm 1, let $v_0, v_1, \dots, v_k$ be the path of nodes visited before the cut separates p at v_k. For every ancestor v_j $(0 \leq j < k)$, p is ultimately inserted as a leaf inside the child subtree of v_j entered at Line 33; since Line 32 updates $B(v_j) \leftarrow B(v_j) \cup \{p\}$ and $M(v_j) \leftarrow M(v_j) + 1$, (16)–(17) are preserved for all $v_0, \dots, v_{k-1}$. At the split locus v_k, the newly allocated parent u_{par} has children v_k and u_{leaf} (where $B(u_{\text{leaf}}) = \{p\}$ and $M(u_{\text{leaf}}) = 1$), and Line 23 sets $B(u_{\text{par}}) = B(v_k) \cup \{p\}$ and $M(u_{\text{par}}) = M(v_k) + 1$, satisfying (16)–(17). During deletion (Algorithm 2), replacing u_{par} with u_{sib} preserves all invariants inside the subtree of u_{sib}, and the upward loop (Lines 18–25) explicitly recomputes (16)–(17) from u_{grand} to r_{root}. □

VII. Experimental Methodology and Verification

A. Verification of Zero-Allocation and Structural Invariants

To empirically verify Theorem 1 and Proposition 1, the engine was subjected to automated invariant and concurrency stress testing under AddressSanitizer (ASan), UndefinedBehaviorSanitizer (UBSan), and ThreadSanitizer (TSan):

1) **Global Heap Interception:** Global memory allocation routines were instrumented with an atomic thread-local interceptor armed prior to entering the streaming loop. Across 10^5 concurrent SPSC ring-buffer transfers, 5×10^4 t-digest quantile updates, and continuous sliding-window RRCF evaluations, the recorded dynamic heap allocation count was verified to be strictly zero.
2) **Degenerate Duplicate-Point Stress Test:** A single tree $(D = 4, W = 64)$ was fed 64 identical constant vectors $p = [5, 5, 5, 5]^\top$, reaching full capacity ($M(r_{\text{root}}) = 64$ and $2W - 1 = 127$ active slab nodes), followed by 64 sequential evictions. Recursive verification of (16)–(17) passed at every step, returning the slab pool cleanly to 0 active nodes.

TABLE I
COMPARISON OF ARCHITECTURAL PROPERTIES ACROSS STREAMING ANOMALY DETECTION PARADIGMS

Architecture	Dynamic Cuts	Phase Space	Hot-Path Allocs	Update Complexity
Batch iForest [3]	No	No	$O(BW)$	$O(BW \log W)$
HS-Trees [4]	No	No	0	$O(B \cdot h_{\max})$
Pointer RRCF [5]	Yes	Opt.	$\geq 2B$	$O(BD \log W)$
LSTM-NDT [2]	N/A	Yes	Var.	$O(L_{\text{seq}} H^2)$
Proposed Engine	**Yes**	**Yes**	**0**	$\mathbf{O(BD \log W)}$

3) **Tail Quantile Fidelity:** Streaming 5×10^4 uniform draws over $[0, 1000]$ through the static merging t-digest ($\delta = 80, C_{\max} = 128, U = 256$) yielded 99th-percentile estimates within $< 1.5\%$ of the analytical inverse cumulative distribution function without dynamic memory growth.

B. Synthetic Dynamical Fault Regimes and Real Telemetry Benchmarks

Table I contrasts the structural properties of the proposed framework against established anomaly detection baselines. To evaluate detection sensitivity across distinct dynamical failure modes, the pipeline ($d = 3, m = 4, B = 32, W = 256, \alpha = 0.995$) was evaluated under two complementary protocols:

1) **Coupled Harmonic Oscillators with Parasitic Flutter Injection:** Three nominal channels were generated as coupled phase-shifted sinusoids corrupted by zero-mean Gaussian noise $\mathcal{N}(0, \sigma^2)$:

$$x_t = \begin{bmatrix} \sin(\omega_0 t) \\ \cos(\omega_0 t) \\ 0.5 \sin(2\omega_0 t) \end{bmatrix} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, 0.04^2 I_3). \tag{18}$$

Periodic 15-frame faults combining high-frequency parasitic flutter ($+4.5 \sin(12\omega_0 t)$ on channel 1) and step bias (-3.8 on channel 3) were injected every 2,000 frames. With phase-space shingling ($m = 4, D = 12$), anomalous frames consistently produced ensemble CoDisp scores exceeding $2\times$ the maximum nominal baseline score and immediately crossed the contamination-gated t-digest threshold $\Theta_t(0.995)$ within a single frame of fault onset.

2) **Memory-Mapped Flight Log Replay:** To support evaluation on recorded avionics benchmarks such as the CMU ALFA UAV fault dataset [1] and NASA spacecraft telemetry [2], the framework provides a POSIX memory-mapped (‘mmap‘) log ingestion utility that standardizes multi-channel CSV traces online via Welford’s algorithm and streams them across the wait-free SPSC shared-memory channel into the detector monitor, maintaining single-digit microsecond per-frame processing latency.

VIII. CONCLUSION

This paper presented a deterministic-memory streaming anomaly detection and telemetry sketching architecture tailored for real-time cyber-physical systems and high-throughput infrastructure. By unifying online moment standardization, Takens’ phase-space delay coordinate embedding, an index-backed zero-allocation Robust Random Cut Forest, and an exponentially decayed merging t-digest over a wait-free cache-aligned shared-memory ring buffer, the proposed framework eliminates dynamic heap allocation in the critical loop while preserving exact theoretical partitioning invariants. Future work will investigate Single Instruction, Multiple Data (SIMD) vectorization of high-dimensional bounding-box span reductions and the integration of incremental topological persistence summaries over the reconstructed phase-space point cloud. The complete source code and benchmark suite are available at GitHub Repository.

REFERENCES

[1] A. Keipour, M. Mousaei, and S. Scherer, “ALFA: A dataset for UAV fault and anomaly detection,” *Int. J. Robot. Res.*, vol. 40, no. 2–3, pp. 513–520, Feb. 2021.

[2] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, “Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding,” in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, London, U.K., Aug. 2018, pp. 387–395.

[3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, Dec. 2008, pp. 413–422.

[4] S. C. Tan, K. M. Ting, and T. F. Liu, “Fast anomaly detection for streaming data,” in *Proc. 22nd Int. Joint Conf. Artif. Intell. (IJCAI)*, Barcelona, Spain, Jul. 2011, pp. 1511–1516.

[5] S. Guha, N. Mishra, G. Roy, and O. Schrijvers, “Robust random cut forest based anomaly detection on streams,” in *Proc. 33rd Int. Conf. Mach. Learn. (ICML)*, New York, NY, USA, Jun. 2016, vol. 48, pp. 2712–2721.

[6] B. P. Welford, “Note on a method for calculating corrected sums of squares and products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, Aug. 1962.

[7] F. Takens, “Detecting strange attractors in turbulence,” in *Dynamical Systems and Turbulence, Warwick 1980*, Lecture Notes in Mathematics, vol. 898. Berlin, Germany: Springer, 1981, pp. 366–381.

[8] T. Sauer, J. A. Yorke, and M. Casdagli, “Embedology,” *J. Stat. Phys.*, vol. 65, no. 3–4, pp. 579–616, Nov. 1991.

[9] T. Dunning, “The t-digest: Efficient estimates of distributions,” *Software Impacts*, vol. 7, Art. no. 100049, Feb. 2021.

[10] T. Dunning and O. Ertl, “Computing extremely accurate quantiles using t-digests,” *arXiv preprint arXiv:1902.04023*, 2019.

[11] L. Lamport, “Specifying concurrent program modules,” *ACM Trans. Program. Lang. Syst.*, vol. 5, no. 2, pp. 190–222, Apr. 1983.

[12] M. Greenwald and S. Khanna, “Space-efficient online computation of quantile summaries,” in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, Santa Barbara, CA, USA, May 2001, pp. 58–66.

[13] J. Bonwick, “The slab allocator: An object-caching kernel memory allocator,” in *Proc. USENIX Summer Tech. Conf.*, Boston, MA, USA, Jun. 1994, pp. 87–98.

[14] J. S. Vitter, “Random sampling with a reservoir,” *ACM Trans. Math. Softw.*, vol. 11, no. 1, pp. 37–57, Mar. 1985.